

Notes on Distributional Reinforcement Learning

Preliminaries

These notes follow the modern treatment of distributional reinforcement learning in [BDR23].

1 Linear Function Approximation

Let $(S_t, A_t, R_{t+1}, S_{t+1})_{t \geq 0}$ be the trajectory of a Markov decision process under a fixed policy π . For a discount factor $\gamma \in [0, 1)$, the return is

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

The classical value function is the conditional mean of this random return:

$$v^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s].$$

Equivalently, it is the fixed point of the Bellman operator

$$(T^\pi v)(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s].$$

In small finite state spaces one may store one value per state. In large or continuous state spaces this is impossible, so we approximate the value function from a feature map

$$\phi : \mathcal{S} \rightarrow \mathbb{R}^d, \quad \phi(s) = \begin{pmatrix} \phi_1(s) \\ \vdots \\ \phi_d(s) \end{pmatrix}.$$

A linear value approximation has the form

$$v_w(s) = \phi(s)^\top w, \quad w \in \mathbb{R}^d.$$

The parameter vector w is shared across states. Thus learning at one state can generalize to other states with similar features.

The temporal-difference error for a transition (S_t, R_{t+1}, S_{t+1}) is

$$\delta_t = R_{t+1} + \gamma \phi(S_{t+1})^\top w_t - \phi(S_t)^\top w_t.$$

The semi-gradient TD(0) update is

$$w_{t+1} = w_t + \alpha_t \delta_t \phi(S_t),$$

where $\alpha_t > 0$ is a step size. This update moves $v_{w_t}(S_t)$ toward the one-step bootstrap target

$$R_{t+1} + \gamma v_{w_t}(S_{t+1}).$$

The important point is that linear TD does not learn the full table $s \mapsto v(s)$. It learns a point in the linear

function class

$$\mathcal{V}_\phi = \{s \mapsto \phi(s)^\top w : w \in \mathbb{R}^d\}.$$

When the true value function is not in this class, the algorithm seeks a projected Bellman fixed point: roughly, the Bellman update is followed by a projection back onto the functions representable by the features. The quality of the approximation therefore depends strongly on the representation ϕ .

2 Distributional Value Approximation

Classical reinforcement learning compresses the return distribution to its mean. Distributional reinforcement learning keeps the random return itself. For a fixed policy π , define

$$Z^\pi(s) \stackrel{D}{=} G_t \mid S_t = s,$$

where $\stackrel{D}{=}$ denotes equality in distribution. Its expectation recovers the classical value:

$$v^\pi(s) = \mathbb{E}[Z^\pi(s)].$$

The distributional Bellman operator is

$$(T^\pi Z)(s) \stackrel{D}{=} R_{t+1} + \gamma Z(S_{t+1}), \quad S_t = s.$$

Thus the Bellman target is no longer a scalar. It is the distribution of a random reward plus a discounted next-state return distribution.

To approximate distributions, one common choice is to represent each return distribution by its quantile function. For a real-valued random variable Z with cumulative distribution function F_Z , its quantile function is

$$F_Z^{-1}(\tau) = \inf\{z \in \mathbb{R} : F_Z(z) \geq \tau\}, \quad \tau \in (0, 1).$$

Choose quantile levels

$$0 < \tau_1 < \dots < \tau_N < 1.$$

A quantile approximation represents the return distribution at state s by

$$Z_\theta(s) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(s)},$$

where δ_x is a Dirac mass at x , and $\theta_i(s)$ is intended to approximate $F_{Z^\pi(s)}^{-1}(\tau_i)$.

With linear function approximation, each quantile location can be parameterized linearly:

$$\theta_i(s) = \phi(s)^\top w_i, \quad w_i \in \mathbb{R}^d.$$

Equivalently, the parameters are

$$W = (w_1, \dots, w_N) \in \mathbb{R}^{d \times N}.$$

This is the distributional analogue of linear value approximation: instead of one linear prediction for the mean return, we use several linear predictions for different parts of the return distribution.

3 Quantile TD

Quantile TD is the temporal-difference idea applied to quantile representations. Given a transition

$$(S_t, R_{t+1}, S_{t+1}),$$

the one-step target samples are

$$y_j = R_{t+1} + \gamma \theta_j(S_{t+1}), \quad j = 1, \dots, N.$$

For the current state, the predictions are

$$x_i = \theta_i(S_t), \quad i = 1, \dots, N.$$

The quantile regression loss for quantile level τ is

$$\rho_\tau(u) = u(\tau - \mathbb{1}_{\{u < 0\}}).$$

For one transition, the distributional TD loss is commonly written as

$$L(W) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}(y_j - x_i).$$

The residual $y_j - x_i$ compares a target quantile sample from the next-state distribution with a predicted quantile at the current state. Minimizing this loss moves the predicted quantiles of $Z_\theta(S_t)$ toward the Bellman-updated distribution

$$R_{t+1} + \gamma Z_\theta(S_{t+1}).$$

For linear quantiles, a subgradient update has the form

$$w_{i,t+1} = w_{i,t} + \alpha_t \left(\frac{1}{N} \sum_{j=1}^N (\tau_i - \mathbb{1}_{\{y_j < x_i\}}) \right) \phi(S_t), \quad i = 1, \dots, N.$$

This mirrors ordinary TD:

$$\text{scalar TD error} \rightsquigarrow \text{quantile regression residuals.}$$

The scalar target $R + \gamma v(S')$ is replaced by a target distribution $R + \gamma Z(S')$, and the mean-squared TD objective is replaced by a quantile regression objective.

In practice, the non-smooth loss ρ_τ is often replaced by a quantile Huber loss, which behaves quadratically near zero and linearly for large residuals. This improves numerical stability while preserving the asymmetric penalty that makes quantile regression estimate quantiles rather than means.

4 Gateway to Deep Representations

The linear setting separates two ideas:

$$\text{features } \phi(s) \quad \text{and} \quad \text{linear prediction heads } w_i.$$

Deep distributional reinforcement learning keeps this structure but learns the features. A neural network first maps the state to a learned representation

$$h_\eta(s) \in \mathbb{R}^d,$$

and then predicts either atom probabilities, quantile locations, or a sampled quantile function from that representation. In the quantile case one may write

$$\theta_i(s) = f_i(h_\eta(s)).$$

This is the conceptual bridge to deep representations: the question is no longer only how to update a fixed set of quantiles, but also what representation makes return distributions easy to predict. The linear model asks, “Which return distributions are expressible using these features?” The deep model asks, “Can the agent learn features for which distributional prediction becomes simple?”

References

- [BDR23] Marc G. Bellemare, Will Dabney, and Mark Rowland. *Distributional Reinforcement Learning*. MIT Press, 2023.